

STAT:5400 Section 3.2 — CLT and Confidence Intervals

Boxiang Wang

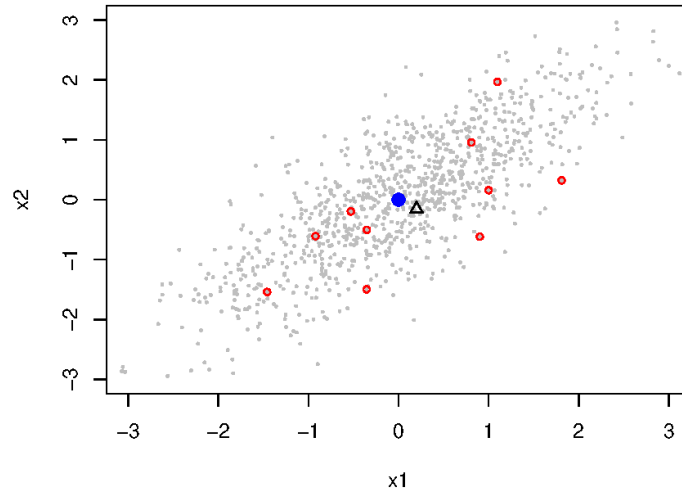
2026-10-06

0.1 Sampling Distribution

0.1.1 Framework of statistical inference

- Data: random **sample** $X_1, \dots, X_n$.
- Model: underlying distribution $N(\mu, \sigma^2)$.
- Assumption: parametric models with unknown **parameters**.
- Goal: estimate **population** parameters μ, σ^2 .
- **Statistic**: $\hat{\mu}$, e.g. the sample mean $\bar{X} = (X_1 + \dots + X_n)/n$, which is a function of the data.

0.1.2 A random sample of 10 observations from a population

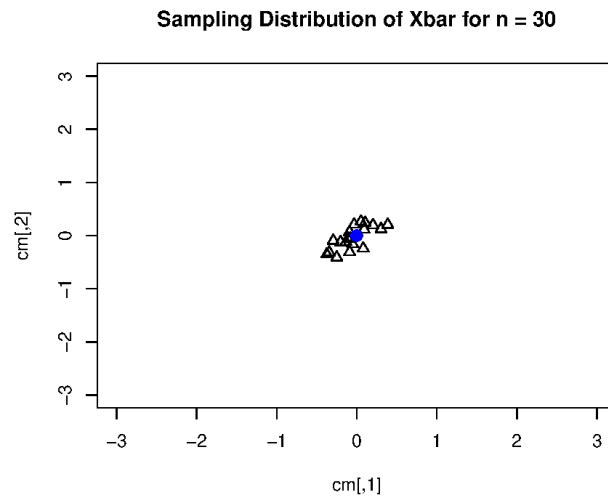
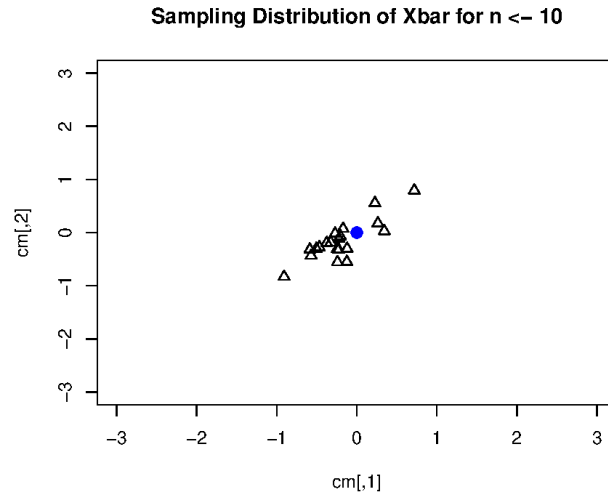


Population: $\mathbf{x} \sim N_2(\mathbf{0}, \Sigma)$, where

$$\Sigma = \begin{bmatrix} 1 & 0.25 \\ 0.25 & 1 \end{bmatrix}.$$

Grey is the population, red is one sample of 10, the black triangle is that sample's mean, and the blue dot is the true mean.

0.1.3 Twenty samples, and the cloud of their means

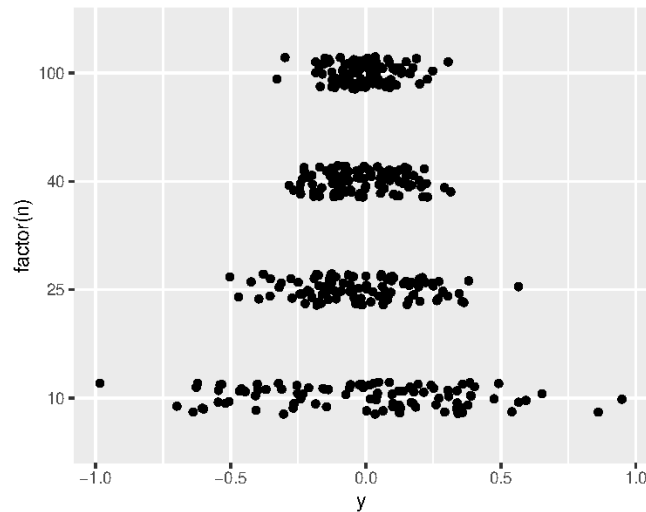


Each triangle is one sample mean. Going from $n = 10$ to $n = 30$ pulls the triangles in toward the truth. That shrinking cloud *is* the sampling distribution.

0.1.4 One dimension, four sample sizes

Run in Codespace — `ggplot2` is not installed in the browser session.

```
n <- rep(c(10, 25, 40, 100), each = 100)
y <- sapply(n, function(n) mean(rnorm(n)))
library(ggplot2)
ggplot() + geom_point(aes(y, factor(n)),
  position = position_jitter(height = 0.15))
```



0.1.5 Distribution of the sample mean for normal populations

Let $X_1, \dots, X_n$ be a random sample from $N(\mu, \sigma^2)$. Let $\bar{X} = n^{-1} \sum_{i=1}^n X_i$. Then

$$\begin{aligned} E(\bar{X}) &= E\left(\frac{X_1 + \dots + X_n}{n}\right) = \frac{1}{n} (EX_1 + \dots + EX_n) = \mu, \\ \text{Var}(\bar{X}) &= \frac{1}{n^2} \text{Var}(X_1 + \dots + X_n) \\ &= \frac{1}{n^2} (\text{Var}(X_1) + \dots + \text{Var}(X_n)) \quad \text{due to independence} \\ &= \sigma^2/n. \end{aligned}$$

Since a linear combination of normal random variables is also normally distributed,

$$\bar{X} \sim N(\mu, \sigma^2/n).$$

0.1.6 Simulating the sampling distribution

```
set.seed(5400)
n <- 30          # sample size
mu <- 1          # population mean
sig.sq <- 4      # population variance

# generating 1000 random samples,
# each of which is of size 30.
# record the 1000 sample means.
Xbarlist <- rep(NA, 1000)
for (i in seq(1000)) {
  Xlist <- rnorm(n, mu, sqrt(sig.sq))
  Xbarlist[i] <- mean(Xlist)
}
```

We used a `for` loop to generate the samples. A faster way is to create a matrix and then compute the row means:

```
# a faster way
set.seed(5400)
n <- 30          # sample size
mu <- 1          # population mean
sig.sq <- 4      # population variance

Xlist <- matrix(rnorm(1000 * n, mu, sqrt(sig.sq)), 1000, n)
Xbarlist <- rowMeans(Xlist)
# may also use apply(Xlist, 1, mean), but slower.
```

0.1.6.1 R

```
hist(Xbarlist)
```

0.1.6.2 Python

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(5400)
```

```

n = 30
mu = 1
sig_sq = 4

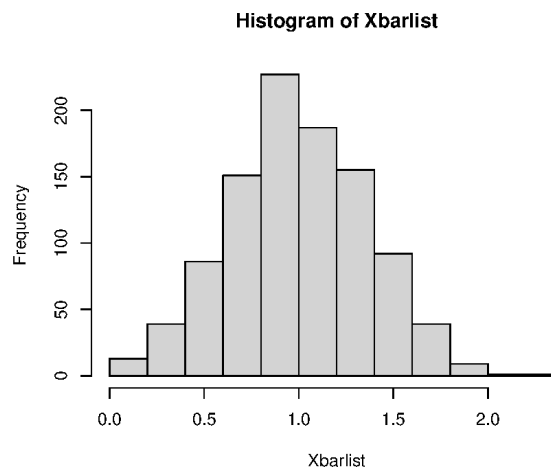
Xbar = [np.mean(np.random.normal(mu, sig_sq**0.5, n))
        for i in range(1000)]

```

```

plt.hist(Xbar)
plt.show()

```



0.1.7 Functions qqnorm and qqline in R

0.1.7.1 R

```

par(mfrow = c(1, 2))
# Q-Q plot by R
qqnorm(Xbarlist, main = "Q-Q plot by R")
qqline(Xbarlist)
# Q-Q plot by hand
probs <- ppoints(1000)
dat_quant <- sort(Xbarlist)
thr_quant <- qnorm(probs)
plot(thr_quant, dat_quant,
     xlab = "Theoretical Quantiles",
     ylab = "Sample Quantiles",

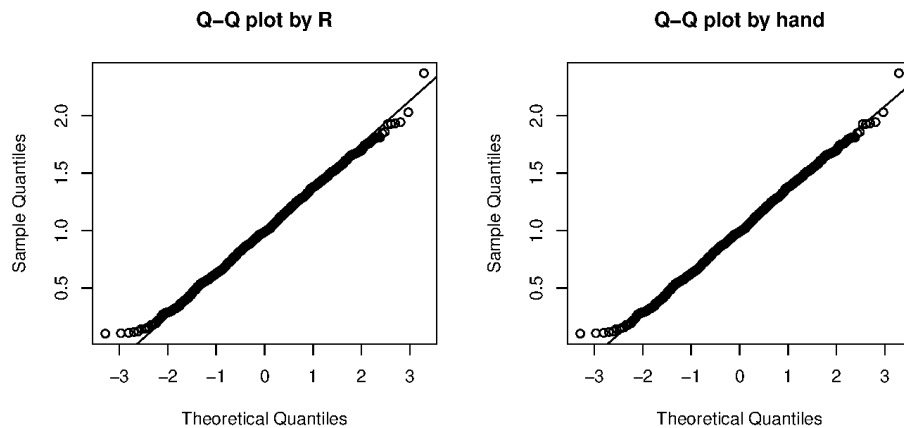
```

```
main = "Q-Q plot by hand")
abline(mean(Xbarlist), sd(Xbarlist))
```

0.1.7.2 Python

```
import scipy.stats

scipy.stats.probplot(np.array(Xbar), plot=plt)
plt.show()
```



The slides use `statsmodels.api.qqplot`. `scipy.stats.probplot` does the same job and is already loaded here.

0.1.8 Sampling distribution

- **Sampling distribution** is the probability distribution of a statistic (a random variable).
- In practice, only one sample is observed. One can investigate the sampling distribution from the theory.

For the sample mean from a random sample with sample size n ,

$$\bar{X} \sim N(\mu, \sigma^2/n).$$

The variance of $\bar{X}$ decreases as the sample size n increases.

0.1.9 Four sample sizes

0.1.9.1 R

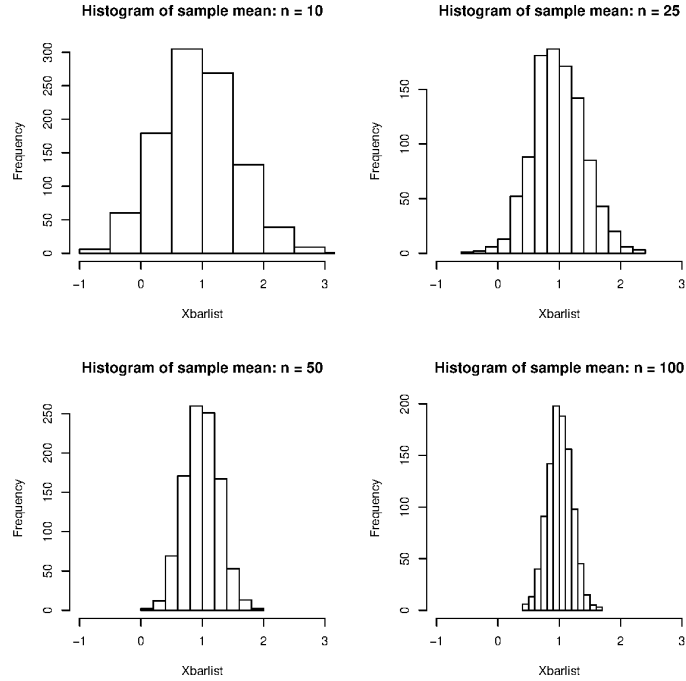
```
set.seed(5400)
mu <- 1          # population mean
sig.sq <- 4      # population variance

par(mfrow = c(2, 2))
for (nlist in c(10, 25, 50, 100)) {
  Xlist <- matrix(rnorm(1000 * nlist,
    mu, sqrt(sig.sq)), 1000, nlist)
  Xbarlist <- rowMeans(Xlist)
  hist(Xbarlist, xlim = c(-1, 3),
    main = paste("Histogram of sample mean: n =", nlist))
}
```

0.1.9.2 Python

```
np.random.seed(5400)
Xbar1 = [np.mean(np.random.normal(mu, sig_sq**0.5, 10)) for i in range(1000)]
Xbar2 = [np.mean(np.random.normal(mu, sig_sq**0.5, 25)) for i in range(1000)]
Xbar3 = [np.mean(np.random.normal(mu, sig_sq**0.5, 50)) for i in range(1000)]
Xbar4 = [np.mean(np.random.normal(mu, sig_sq**0.5, 100)) for i in range(1000)]
```

```
fig, axs = plt.subplots(2, 2)
fig.suptitle('Histogram of sample mean of normal dist')
axs[0, 0].hist(Xbar1); axs[0, 0].set_title('n = 10')
axs[0, 1].hist(Xbar2); axs[0, 1].set_title('n = 25')
axs[1, 0].hist(Xbar3); axs[1, 0].set_title('n = 50')
axs[1, 1].hist(Xbar4); axs[1, 1].set_title('n = 100')
for ax in axs.flat:
    ax.set(xlabel='Xbarlist', ylabel='Frequency')
for ax in axs.flat:
    ax.label_outer()
plt.show()
```



0.2 Central Limit Theorem

0.2.1 Classical review of the central limit theorem

Let $X_1, \dots, X_n$ be a random sample drawn from a distribution with mean $EX = \mu$ and variance $\text{Var}(X) = \sigma^2 < \infty$. Let $\bar{X} = n^{-1} \sum_{i=1}^n X_i$.

The classical **central limit theorem** states that

$$\frac{\sqrt{n}(\bar{X} - \mu)}{\sigma} \xrightarrow{\mathcal{D}} N(0, 1),$$

which is equivalent to saying

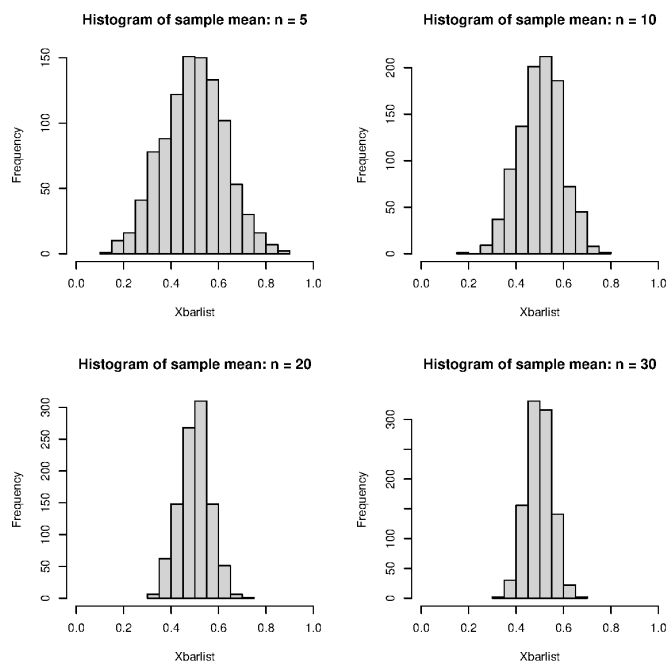
$$\bar{X} \text{ approximately follows } N(\mu, \sigma^2/n),$$

if the sample size n is sufficiently large.

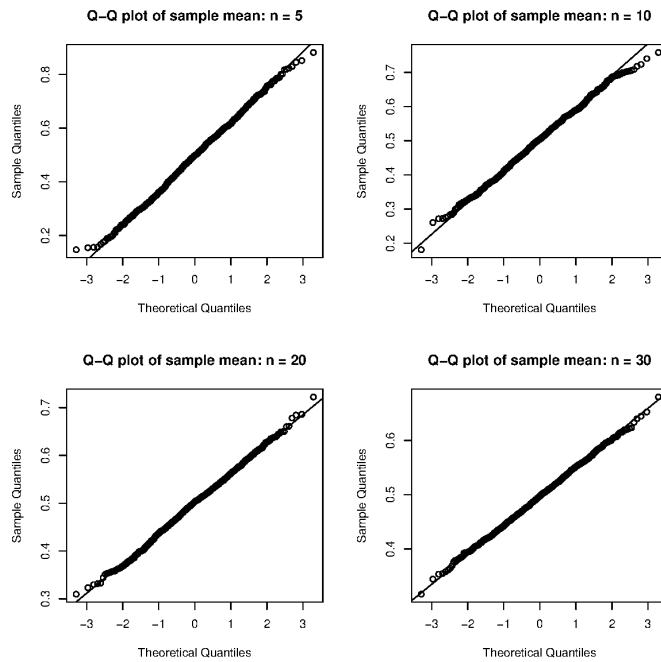
The population no longer has to be normal. That is the whole content of the theorem, and the next two examples are the demonstration.

0.2.2 CLT for random samples drawn from the uniform

```
set.seed(5400)
par(mfrow = c(2, 2))
for (nlist in c(5, 10, 20, 30)) {
  Xlist <- matrix(runif(1000 * nlist), 1000, nlist)
  Xbarlist <- rowMeans(Xlist)
  hist(Xbarlist, xlim = c(0, 1),
       main = paste("Histogram of sample mean: n =", nlist))
}
```

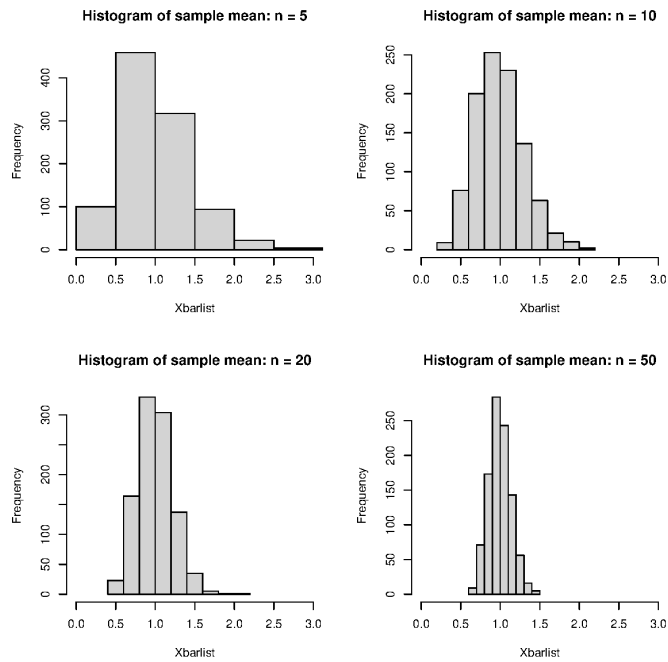


```
set.seed(5400)
par(mfrow = c(2, 2))
for (nlist in c(5, 10, 20, 30)) {
  Xlist <- matrix(runif(1000 * nlist), 1000, nlist)
  Xbarlist <- rowMeans(Xlist)
  qqnorm(Xbarlist,
        main = paste("Q-Q plot of sample mean: n =", nlist))
  qqline(Xbarlist)
}
```

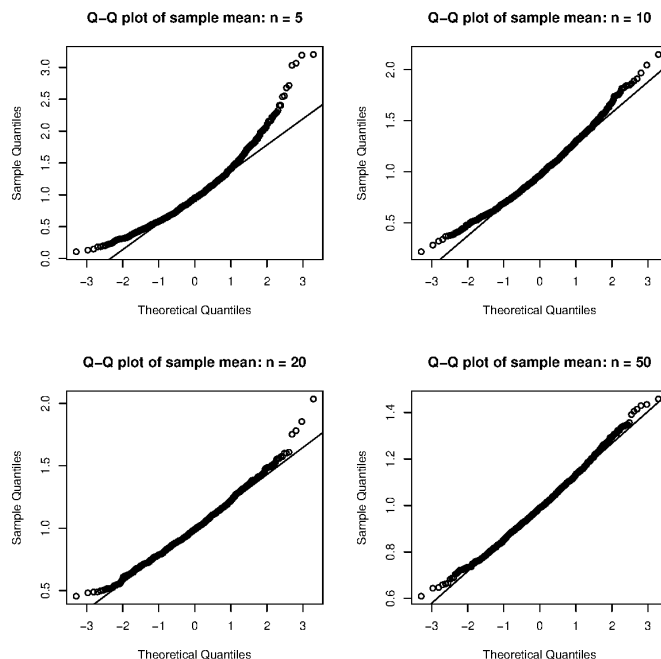


0.2.3 CLT for random samples drawn from the exponential

```
set.seed(5400)
par(mfrow = c(2, 2))
for (nlist in c(5, 10, 20, 50)) {
  Xlist <- matrix(rexp(1000 * nlist), 1000, nlist)
  Xbarlist <- rowMeans(Xlist)
  hist(Xbarlist, xlim = c(0, 3),
       main = paste("Histogram of sample mean: n =", nlist))
}
```



```
set.seed(5400)
par(mfrow = c(2, 2))
for (nlist in c(5, 10, 20, 50)) {
  Xlist <- matrix(rexp(1000 * nlist), 1000, nlist)
  Xbarlist <- rowMeans(Xlist)
  qqnorm(Xbarlist,
    main = paste("Q-Q plot of sample mean: n =", nlist))
  qqline(Xbarlist)
}
```



💡 Predict — the uniform is symmetric and the exponential is not. Which one needs a bigger n before the Q-Q plot straightens out?

i Which converges faster

The uniform, by a wide margin. Its sample mean already looks normal at $n = 5$; the exponential still shows a right-skewed bend at $n = 20$.

The reason is skewness. A Berry-Esseen style bound says the error in the normal approximation decays like $\gamma/\sqrt{n}$, where γ is the population skewness. The uniform has $\gamma = 0$, the exponential has $\gamma = 2$.

This is why “ $n > 30$ is enough” is folklore, not a theorem. It depends on the shape you started from.

0.3 Confidence Interval

0.3.1 Case I: known variance, normality

- Under **normality**, the distribution of the sample mean $\bar{X}$ is $\bar{X} \sim N(\mu, \sigma^2/n)$.
- Due to the normality of $\bar{X}$, we can standardize:

$$Z = \frac{\bar{X} - \mu}{\sigma/\sqrt{n}} \sim N(0, 1).$$

- Thus

$$\begin{aligned} 1 - \alpha &= P(z_{1-\alpha/2} < Z < z_{\alpha/2}) \\ &= P\left(z_{1-\alpha/2} < \frac{\bar{X} - \mu}{\sigma/\sqrt{n}} < z_{\alpha/2}\right) \\ &= P(\bar{X} - z_{\alpha/2}\sigma/\sqrt{n} < \mu < \bar{X} + z_{\alpha/2}\sigma/\sqrt{n}). \end{aligned}$$

0.3.2 Getting the quantile

Denote by z_u the 100 u th percentile of the standard normal distribution, so $z_u = -z_{1-u}$. In R and Python, to compute $z_{0.025}$:

0.3.2.1 R

```
qnorm(0.025, lower = FALSE) # default: lower tail
-qnorm(0.025)                # same result
pnorm(1.96)                  # cdf
```

0.3.2.2 Python

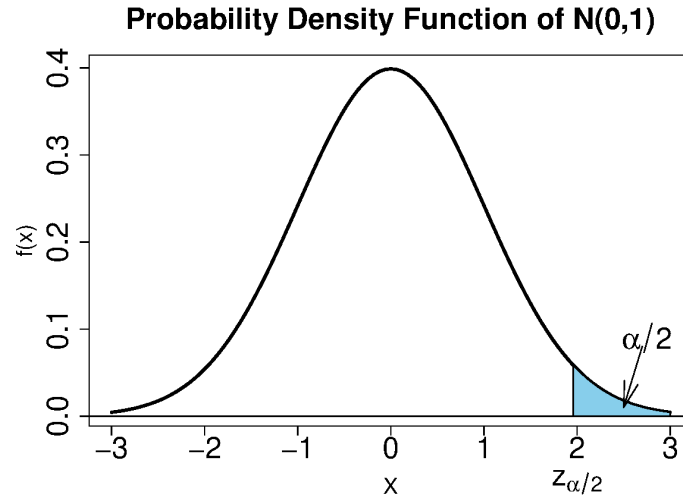
```
import scipy.stats

print(-scipy.stats.norm.ppf(0.025))
print(scipy.stats.norm.cdf(1.96))
```

R's `qnorm` and Python's `scipy.stats.norm.ppf` are the same function under two names. Python has no `lower=FALSE` switch, so you negate or use `isf`.

```
curve(dnorm(x, 0, 1), xlim = c(-3, 3),
      main = "Probability Density Function of N(0,1)",
      xlab = "X", ylab = "f(x)", lwd = 3)
cord.x <- c(qnorm(0.975), seq(qnorm(0.975), 3, 0.01), 3)
cord.y <- c(0, dnorm(seq(qnorm(0.975), 3, 0.01)), 0)
polygon(cord.x, cord.y, col = "skyblue")
mtext(expression(paste(z[alpha/2])), at = 2, line = 3, side = 1)
```

```
arrows(2.7, 0.08, 2.5, 0.01, angle = 15, length = 0.15)
text(2.75, 0.09, expression(alpha/2))
abline(h = 0)
```



0.3.3 Standard error versus standard deviation

When σ is known, the confidence interval of the mean is $\bar{X} \pm z_{\alpha/2}\sigma/\sqrt{n}$:

- We call $\sigma/\sqrt{n}$ the **standard error** of $\bar{X}$, which is the standard deviation of the sampling distribution of $\bar{X}$.
- Note that σ is the **standard deviation** of X .

When σ is unknown, the sample standard deviation S is used to estimate σ . The confidence interval is $\bar{X} \pm z_{\alpha/2}S/\sqrt{n}$.

- The **standard error** $S/\sqrt{n}$ is the estimate of the standard deviation of the sampling distribution of $\bar{X}$.

0.3.4 Coverage probability of a random confidence interval

- For simplicity, we assume the variance σ^2 is known.
- In practice, we observe only one sample, and give a confidence interval $\bar{X} \pm z_{\alpha/2}\sigma/\sqrt{n}$. We are $100(1 - \alpha)\%$ confident that this interval contains the true parameter μ .
- The confidence means the coverage probability, which is the probability of the interval containing the true parameter under the **sampling distribution**.

- Now we use a simulation study to simulate the sampling distribution and assess the coverage probability.

0.3.4.1 R

```
# case I true dist: N(0.5, 1)
set.seed(5400)
n <- 20
alp <- 0.05
sig.sq <- 1
mu <- 0.5
Xlist <- matrix(rnorm(10000 * n, mu, sqrt(sig.sq)), 10000, n)
Xbarlist <- rowMeans(Xlist)

moe <- qnorm(1 - alp/2) * sqrt(sig.sq / n)
CI <- cbind(Xbarlist - moe, Xbarlist + moe)
is_cover <- apply(CI, 1, function(x) mu > x[1] & mu < x[2])
mean(is_cover)
```

0.3.4.2 Python

```
np.random.seed(5400)
n = 20
alp = 0.05
sig_sq = 1
mu = 0.5
Xlist = [np.random.normal(mu, sig_sq**0.5, n) for i in range(10000)]
Xbarlist = np.mean(Xlist, 1)

cri = scipy.stats.norm.ppf(1 - alp/2)
moe = cri * (sig_sq / n)**0.5
print(np.mean([(mu > Xbarlist[i] - moe and mu < Xbarlist[i] + moe)
               for i in range(10000)]))
```

The answer is near 0.95, as it should be. Everything after this section is a study of when it stops being near 0.95.

0.3.5 Case II: unknown variance, normality

The normality of X gives

$$\frac{\bar{X} - \mu}{\sigma/\sqrt{n}} \sim N(0, 1).$$

By replacing σ by its estimator S we have

$$\frac{\bar{X} - \mu}{S/\sqrt{n}} \sim t_{n-1},$$

where $S^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2$ is the sample variance.

Question: Why does $T = (\bar{X} - \mu)/(S/\sqrt{n})$ follow t_{n-1} ?

Answer: Student's theorem. If $X_1, \dots, X_n \stackrel{\text{iid}}{\sim} N(\mu, \sigma^2)$, then

1. $(\bar{X} - \mu)/(\sigma/\sqrt{n}) \sim N(0, 1)$;
2. $(n-1)S^2/\sigma^2 \sim \chi_{n-1}^2$;
3. $\bar{X}$ and S^2 are independent.

Recall the t -distribution: $Z/\sqrt{X/v} \sim t_v$, where $Z \sim N(0, 1)$, $X \sim \chi_v^2$, and Z, X are independent. We have

$$T = \frac{\bar{X} - \mu}{S/\sqrt{n}} = \frac{\frac{\bar{X} - \mu}{\sigma/\sqrt{n}}}{\sqrt{\frac{(n-1)S^2}{\sigma^2}} / (n-1)} \sim t_{n-1}.$$

- The t -distribution with **degrees of freedom** $n-1$ has heavier tails than $N(0, 1)$.
- Thus we still have

$$\begin{aligned} 1 - \alpha &= P\left(t_{1-\alpha/2, n-1} < \frac{\bar{X} - \mu}{S/\sqrt{n}} < t_{\alpha/2, n-1}\right) \\ &= P(\bar{X} - t_{\alpha/2, n-1} S/\sqrt{n} < \mu < \bar{X} + t_{\alpha/2, n-1} S/\sqrt{n}). \end{aligned}$$

- The distribution t_n converges to $N(0, 1)$ as $n \rightarrow \infty$.

```

curve(dnorm(x, 0, 1), xlim = c(-3, 3), xlab = "X",
      ylab = "f(x)", lwd = 3, lty = 1, main = "Normal and t")
xl <- c(-3, 3)
curve(dt(x, 1), xlim = xl, add = TRUE, lty = 2, col = "red")
curve(dt(x, 2), xlim = xl, add = TRUE, lty = 3, col = "blue")
curve(dt(x, 5), xlim = xl, add = TRUE, lty = 4, col = "green")
curve(dt(x, 50), xlim = xl, add = TRUE, lty = 5, col = "brown")
abline(h = 0)
legend("topright",
      c("normal", "t(1)", "t(2)", "t(5)", "t(50)"),
      lty = 1:5,
      col = c("black", "red", "blue", "green", "brown"))

```

👉 By hand — fill in the margin of error so this interval uses the t quantile and the *estimated* standard deviation. Done when the coverage prints near 0.95. No AI for this block.

```

# case II true dist: N(0.5, 1)
set.seed(5400)
n <- 20; alp <- 0.05; mu <- 0.5; sig.sq <- 1
Xlist <- matrix(rnorm(10000 * n, mu, sqrt(sig.sq)), 10000, n)
Xbarlist <- rowMeans(Xlist)
sdlist <- apply(Xlist, 1, sd)

se <- 0                                # <- change this line

CI <- cbind(Xbarlist - se, Xbarlist + se)
is_cover <- apply(CI, 1, function(x) mu > x[1] & mu < x[2])
mean(is_cover)

```

Solution

```
se <- qt(1 - alp/2, n - 1) * sdlist / sqrt(n)
```

This gives about 0.9504. Two things changed from case I: the quantile is `qt` with $n - 1$ degrees of freedom rather than `qnorm`, and the width now varies from row to row, because each simulated sample has its own S .

If you use `qnorm` here instead, coverage drops to roughly 0.93. The t quantile is wider, and that extra width is exactly what pays for having estimated σ .

0.3.6 Case III: non-normality, large sample

- When the sample size is large, $n > 30$ or $n > 50$, by the CLT the distribution of the sample mean $\bar{X}$ is $\bar{X} \sim N(\mu, \sigma^2/n)$.
- Thus we still have

$$Z = \frac{\bar{X} - \mu}{\sigma/\sqrt{n}} \sim N(0, 1),$$
$$1 - \alpha = P(\bar{X} - z_{\alpha/2}\sigma/\sqrt{n} < \mu < \bar{X} + z_{\alpha/2}\sigma/\sqrt{n}).$$

- If the variance is unknown, since n is large, S is a consistent estimator of σ . With Slutsky's theorem,

$$Z = \frac{\bar{X} - \mu}{S/\sqrt{n}} \sim N(0, 1),$$
$$1 - \alpha = P(\bar{X} - z_{\alpha/2}S/\sqrt{n} < \mu < \bar{X} + z_{\alpha/2}S/\sqrt{n}).$$

- We will investigate how large n should be in Section 3.3.

👉 By hand — fill in the margin of error for case III. The population is now exponential, not normal. Done when the coverage prints and is clearly below 0.95. No AI for this block.

```
# case III true dist: exp with mean 2
set.seed(5400)
n <- 20; alp <- 0.05; mu <- 2
Xlist <- matrix(rexp(10000 * n, rate = 1/mu), 10000, n)
Xbarlist <- rowMeans(Xlist)
sdlist <- apply(Xlist, 1, sd)

moe <- 0                                # <- change this line

CI <- cbind(Xbarlist - moe, Xbarlist + moe)
is_cover <- apply(CI, 1, function(x) mu > x[1] & mu < x[2])
mean(is_cover)
```

Solution

```
moe <- qnorm(1 - alp/2) * sdlist / sqrt(n)
```

Coverage comes out near 0.90, not 0.95. Nothing is wrong with the code. The interval is behaving exactly as the theory says it should when n is too small for the CLT to have taken hold, and the population is skewed.

An interval that claims 95 percent and delivers 90 percent is not a rounding problem. In a real study it means one in ten of your published intervals misses, not one in twenty.

0.3.7 How large is large enough?

0.3.7.1 R

```
CovProb <- function(n, mu = 2, alp = 0.05) {  
  Xlist <- matrix(rexp(10000 * n, rate = 1/mu), 10000, n)  
  Xbarlist <- rowMeans(Xlist)  
  sdlist <- apply(Xlist, 1, sd)  
  moe <- qnorm(1 - alp/2) * sdlist / sqrt(n)  
  CI <- cbind(Xbarlist - moe, Xbarlist + moe)  
  is_cover <- apply(CI, 1, function(x) mu > x[1] & mu < x[2])  
  mean(is_cover)  
}
```

```
set.seed(5400)  
CovProb(10)  
CovProb(30)  
CovProb(50)  
CovProb(200)
```

0.3.7.2 Python

```
def CovProb(n, mu=2, alp=0.05):  
    Xlist = [np.random.exponential(mu, n) for i in range(10000)]  
    Xbarlist = np.mean(Xlist, 1)  
    Slist = np.std(Xlist, 1, ddof=1)  
    moe = scipy.stats.norm.ppf(1 - alp/2) * Slist / n**0.5  
    return np.mean([(mu > Xbarlist[i] - moe[i] and  
                    mu < Xbarlist[i] + moe[i])  
                    for i in range(10000)])
```

```
np.random.seed(5400)  
print(CovProb(10))  
print(CovProb(30))  
print(CovProb(50))  
print(CovProb(200))
```

Even at $n = 200$ the coverage has not quite reached 0.95. Convergence is real but slow.

0.3.8 Violation of the independence assumption

- The confidence intervals that we have computed fail if the independence assumption of $X_1, \dots, X_n$ is violated.
- We assume each of $X_1, \dots, X_n$ follows $N(0, 1)$ but

$$\text{Cov}(X_1, \dots, X_n) = \Sigma = \begin{pmatrix} 1 & \rho & \dots & \rho \\ \rho & 1 & \dots & \rho \\ \vdots & \vdots & \ddots & \vdots \\ \rho & \rho & \dots & 1 \end{pmatrix},$$

where $\Sigma_{ij} = \text{Cov}(X_i, X_j)$.

- Assume $\sigma = 1$ is known; we compute confidence intervals as $\bar{X} \pm z_{\alpha/2} \sigma / \sqrt{n}$. We investigate the coverage probability of $\mu = 0$ as ρ varies.

0.3.9 Generating dependent samples

First, we compute $\Sigma^{1/2}$ such that $\Sigma^{1/2} \Sigma^{1/2} = \Sigma$.

```
rho <- 0.1; n <- 5
Sigma <- matrix(rho, n, n)
diag(Sigma) <- 1
eig <- eigen(Sigma)
Sigma.sq <- eig$vectors %*%
  tcrossprod(diag(sqrt(eig$values)), eig$vectors)
```

```
Sigma.sq %*% Sigma.sq
```

We generate a dependent sample $X_1, \dots, X_5$ such that $\text{Cov}(X_i, X_j) = 0.1$ for $i \neq j$.

```
set.seed(5400)
X <- rnorm(n) %*% Sigma.sq
X
```

The same code was used in Section 3.1 to generate an i.i.d. five-dimensional multivariate normal, but the two tasks are conceptually different. There, five numbers were one observation of a vector. Here, five numbers are five correlated observations of a scalar.

How about many samples at once?

```

set.seed(5400)
bigN <- 1e4 # the slides use 1e5; 1e4 keeps this quick in a browser
Xlist <- matrix(rnorm(bigN * n, 0, 1), bigN, n) %*% Sigma.sq
round(var(Xlist), 3)

```

The off-diagonal entries come out near 0.1, which is what we asked for.

0.3.10 Coverage as ρ varies

```

# bigN is 1e5 in the slides; 1e4 keeps the 20-point curve quick in a browser
CovProb2 <- function(rho, n = 30, mu = 0, bigN = 1e4, alp = 0.05) {
  Sigma <- matrix(rho, n, n)
  diag(Sigma) <- 1
  eig <- eigen(Sigma)
  Sigma.sq <- eig$vectors %*%
    tcrossprod(diag(sqrt(eig$values)), eig$vectors)
  # generate dependent samples
  Xlist <- matrix(rnorm(bigN * n, mu, 1), bigN, n) %*% Sigma.sq
  Xbarlist <- rowMeans(Xlist)
  moe <- qnorm(1 - alp/2) * 1 / sqrt(n)
  CI <- cbind(Xbarlist - moe, Xbarlist + moe)
  is_cover <- apply(CI, 1, function(x) mu > x[1] & mu < x[2])
  mean(is_cover)
}

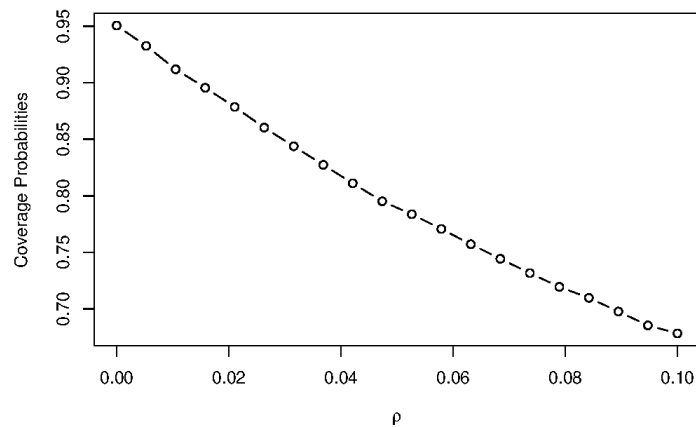
```

```

set.seed(5400)
rhos <- seq(0, 0.1, len = 20)
covprobs <- rep(NA, 20)
for (i in seq(20)) {
  # call the function above
  covprobs[i] <- CovProb2(rhos[i])
}

plot(rhos, covprobs, type = "b",
     xlab = expression(rho),
     ylab = "Coverage Probabilities")

```



The code appears better (but not faster) with the `sapply` function:

```
set.seed(5400)
covprobs <- sapply(rhos, CovProb2)
head(covprobs)
```

as compared with

```
covprobs <- rep(NA, 20)
for (i in seq(20)) {
  covprobs[i] <- CovProb2(rhos[i])
}
```

💡 Predict — $\rho = 0.1$ is a small correlation. Look at the plot. How far does coverage fall by the time ρ reaches 0.1?

i Going further: why a tiny correlation costs so much

Coverage falls from 0.95 to roughly 0.80 by $\rho = 0.1$.

The reason is that the variance of the mean under equicorrelation is

$$\text{Var}(\bar{X}) = \frac{\sigma^2}{n} [1 + (n-1)\rho].$$

With $n = 30$ and $\rho = 0.1$ the bracket is $1 + 29(0.1) = 3.9$. The true standard error is almost twice what the formula assumes, so the interval is about half as wide as it should be.

Notice the n in the bracket. A larger sample makes dependence *worse*, not better. This is why cluster-randomized trials report a design effect, and why treating repeated measures as independent is one of the most expensive mistakes in applied work.

0.4 Confidence Interval for a Proportion

0.4.1 Normal approximation of Bernoulli

- Suppose $X_1, \dots, X_n$ are drawn from $\text{Bern}(\theta)$.
- If n is sufficiently large, from the classical CLT, $\bar{X} \sim N(\theta, \theta(1-\theta)/n)$.
- Some textbooks suggest $\min\{n\theta, n(1-\theta)\} \geq 15$ makes such an approximation good enough.
- Equivalently,

$$\frac{\bar{X} - \theta}{\sqrt{\theta(1-\theta)/n}} \sim N(0, 1), \text{ approximately.}$$

- For $\alpha \in (0, 1)$,

$$\begin{aligned} 1 - \alpha &\approx P\left(-z_{\alpha/2} < \frac{\bar{X} - \theta}{\sqrt{\theta(1-\theta)/n}} < z_{\alpha/2}\right) \\ &= P\left(\bar{X} - z_{\alpha/2}\sqrt{\frac{\theta(1-\theta)}{n}} < \theta < \bar{X} + z_{\alpha/2}\sqrt{\frac{\theta(1-\theta)}{n}}\right). \end{aligned}$$

0.4.2 Six intervals for a proportion

- The **Wald** $100(1-\alpha)\%$ confidence interval:

$$\bar{X} \pm z_{\alpha/2} \sqrt{\frac{\bar{X}(1-\bar{X})}{n}}.$$

- The **simple conservative** $100(1-\alpha)\%$ confidence interval:

$$\bar{X} \pm z_{\alpha/2} \frac{0.5}{\sqrt{n}}.$$

- The **Wilson** (score) $100(1-\alpha)\%$ confidence interval:

$$\frac{n\bar{X} + 0.5z_{\alpha/2}^2}{n + z_{\alpha/2}^2} \pm \frac{z_{\alpha/2}\sqrt{n\bar{X}(1-\bar{X}) + 0.25z_{\alpha/2}^2}}{n + z_{\alpha/2}^2}.$$

This interval can be obtained in R with

```
prop.test(x = n * xbar, n = n, conf.level = 1 - alp, correct = FALSE)$conf.int
```

- The **Clopper-Pearson** $100(1-\alpha)\%$ interval has left endpoint

$$\left(1 + \frac{n - n\bar{X} + 1}{n\bar{X}F_{1-\alpha/2, 2n\bar{X}, 2(n-n\bar{X}+1)}}\right)^{-1},$$

and right endpoint

$$\left(1 + \frac{n - n\bar{X}}{(n\bar{X} + 1)F_{\alpha/2, 2(n\bar{X}+1), 2(n-n\bar{X})}}\right)^{-1}.$$

Here F_{u, v_1, v_2} is the $100(1-u)$ th percentile of the F -distribution with degrees of freedom v_1 and v_2 . In R it is `qf(u, v1, v2)`. The interval is obtained with

```
binom.test(x = n * xbar, n = n, conf.level = 1 - alp)$conf.int
```

- The **Agresti-Coull** $100(1-\alpha)\%$ confidence interval:

$$\hat{p} \pm z_{\alpha/2}\sqrt{\frac{1}{n + z_{\alpha/2}^2}\hat{p}(1-\hat{p})}, \quad \hat{p} = \bar{X} + \frac{z_{\alpha/2}^2}{2(n + z_{\alpha/2}^2)}.$$

```
library(binom)
binom.confint(x = n * xbar, n = n, conf.level = 1 - alp, methods = "ac")
```

- The **Jeffreys** $100(1-\alpha)\%$ confidence interval: the $100(1-\alpha)\%$ Bayesian credible interval using Jeffreys' prior on the binomial proportion.

```
library(binom)
binom.bayes(x = n * xbar, n = n, type = "central", conf.level = 1 - alp)
```

0.4.3 Comparing the six

Our goal is to compare the differences among the six confidence intervals in terms of coverage probability and expected width. We use the same generated data for all six methods.

- **Tip 1:** Always report the **standard error** of entries in the table so a reader can gauge the accuracy.
- **Tip 2:** Since humans cannot understand more than two digits very easily, **round** your results to two or three digits.

References:

- Brown, Cai, and DasGupta (2001) *Interval Estimation for a Binomial Proportion*. **Statistical Science**. 16(2). 101–133.
- Wikipedia: https://en.wikipedia.org/wiki/Binomial_proportion_confidence_interval
- binom package: <https://cran.r-project.org/web/packages/binom/binom.pdf>

```
set.seed(5400)
theta <- 0.6
n <- 40
B <- 1000          # the slides use B = 10000; 1000 keeps this quick in a browser
alp <- 0.05
# generate data
Xbarlist <- rbinom(B, size = n, prob = theta) / n
```

```
## Wald interval
moe_Wald <- qnorm(1 - alp/2) * sqrt(Xbarlist * (1 - Xbarlist)/n)
CI_Wald <- cbind(Xbarlist - moe_Wald, Xbarlist + moe_Wald)
## simple conservative interval
moe_simple <- qnorm(1 - alp/2) * sqrt(0.5 * (1 - 0.5)/n)
CI_simple <- cbind(Xbarlist - moe_simple, Xbarlist + moe_simple)
```

```
library(binom)
## score, CP, AC, and the Jeffreys Bayesian CIs
CI_score <- CI_CP <- CI_AC <- CI_Bayes <- matrix(NA, B, 2)
for (i in seq(B)) {
  CI_score[i, ] <- prop.test(x = (n * Xbarlist[i]), n = n,
    correct = FALSE, conf.level = (1 - alp))$conf.int
  CI_CP[i, ] <- binom.test(x = (n * Xbarlist[i]), n = n,
    conf.level = (1 - alp))$conf.int
  ret <- binom.confint(x = (n * Xbarlist[i]), n = n,
    conf.level = (1 - alp), methods = "ac")
  CI_AC[i, ] <- c(ret$lower, ret$upper)
```

```

ret2 <- binom.bayes(x = (n * Xbarlist[i]), n = n,
  type = "central", conf.level = (1 - alp))
CI_Bayes[i, ] <- c(ret2$lower, ret2$upper)
}

## get mean and se of width and coverage probability
WidthCov <- function(CI) {
  # get coverage probability
  is_cov <- function(x) theta > x[1] & theta < x[2]
  # get standard error
  se <- function(x) sd(x) / sqrt(nrow(CI))
  allwidth <- CI[, 2] - CI[, 1]      # width
  allcov <- apply(CI, 1, is_cov)     # coverage
  c(mean(allwidth), se(allwidth), mean(allcov), se(allcov))
}

```

Based on the current simulation result, which method is favorable?

```

## display results
col.nm <- paste0("CI_", c("simple", "Wald", "score", "CP", "AC", "Bayes"))
res <- sapply(lapply(col.nm, get), WidthCov)
colnames(res) <- col.nm
rownames(res) <- c("exp.width", "se", "cov.prob", "se")
round(res, 3)

```

Run in Codespace — the same comparison at the full $B = 10000$ of the slides.

i Reading the table

At this one setting, Wald looks fine. The exact coverages at $\theta = 0.6$, $n = 40$ are 0.946 for Wald and for Jeffreys, and 0.966 for the other four. Wald is the narrowest, and its coverage is the closest to the nominal 0.95.

That is the trap. Coverage of a binomial interval is not a single number; it is a function of θ , and for Wald that function is erratic:

θ	0.02	0.05	0.10	0.20	0.50	0.60
Wald	0.553	0.868	0.914	0.905	0.919	0.946
Wilson	0.954	0.952	0.943	0.928	0.962	0.966

Over $\theta \in (0.01, 0.99)$, Wald averages 0.905 coverage and dips as low as 0.33. Wilson averages 0.952 and never falls below 0.905, at a width no larger than Wald's. That is why Brown, Cai and DasGupta recommend against Wald despite tables like the one above. Change `theta` to 0.05 and run the four blocks again. Then look at the `se` rows: with $B = 1000$ the standard error of a coverage estimate is about 0.007, so differences under 0.01 are not real.

0.4.4 Python version

```
np.random.seed(5400)
theta = 0.6; n = 40; B = 1000; alp = 0.05
Xbarlist = np.array([np.random.binomial(n, theta) / n for i in range(B)])
```

```
## Wald interval
cri_Wald = scipy.stats.norm.ppf(1 - alp/2)
moe_Wald = cri_Wald * np.sqrt(Xbarlist * (1 - Xbarlist) / n)
CI_Wald = np.c_[Xbarlist - moe_Wald, Xbarlist + moe_Wald]
```

```
## simple conservative interval
cri_simple = scipy.stats.norm.ppf(1 - alp/2)
moe_simple = cri_simple * np.sqrt(0.5 * (1 - 0.5) / n)
CI_simple = np.c_[Xbarlist - moe_simple, Xbarlist + moe_simple]
```

```
def width_cov(CI):
    w = CI[:, 1] - CI[:, 0]
    cov = (theta > CI[:, 0]) & (theta < CI[:, 1])
    return [w.mean(), w.std(ddof=1) / np.sqrt(len(w)),
            cov.mean(), cov.std(ddof=1) / np.sqrt(len(cov))]

print(np.round(width_cov(CI_simple), 3))
print(np.round(width_cov(CI_Wald), 3))
```

Python has no single package that matches `binom` for all six methods. `statsmodels.stats.proportion.proportion` covers Wald, Wilson, Agresti-Coull, and Clopper-Pearson (`method="beta"`), which is four of the six.